

Camera-based Human Activity Recognition: Comparative analysis of Deep Learning approaches

Marteinn Hjaltason
*Dept. of Computer Science
and Media Technology
Malmö University*
Email: marteinn100@hotmail.com

Wilmer H. Jacobsson
*Dept. of Computer Science
and Media Technology
Malmö University*
Email: wilmerhjacobsson@gmail.com

João Domingos
*Dept. of Computer Science
and Media Technology
Malmö University*
Email: jndomingos1998@gmail.com

Abstract—Human Activity Recognition (HAR) aims to automate the classification of human actions from sensory inputs, offering significant potential for real-world applications such as security and healthcare surveillance. However, deploying HAR systems in continuous video settings presents a significant challenge, leveraging the trade-off between recognition accuracy and computational efficiency under low-latency constraints. This study investigates the suitability of different deep learning architectures for practical, real-time edge deployment by classifying atomic human activities from RGB video inputs. This project conducts a comparative analysis of three distinct architectures, a lightweight 2D CNN-LSTM hybrid (EfficientNet-B0), a separable 3D CNN (S3D), and a Multiscale Vision Transformer (MViT v2), using the traditional benchmark KTH Action Recognition dataset. The models were evaluated based on classification accuracy, parameter count, floating-point operations per second (FLOPS), and inference time. The results reveal that the S3D architecture provides the optimal speed-accuracy trade-off, achieving the highest accuracy of 96.5% while having a manageable computational efficiency of 18.20 GFLOPS. In contrast, the EfficientNet-LSTM was the lightest model (6.41 GFLOPS) but lagged in accuracy (89%), while the MViT v2's massive computational demand (64.46 GFLOPS) suggests its incompatibility with compute-constrained environments despite reaching 92.7% accuracy. Ultimately, S3D is recommended as the most robust and efficient approach for real-time video HAR applications.

Keywords: Human Activity Recognition, Deep Learning, Video Classification, CNN-LSTM, S3D, Vision Transformers, Real-Time Inference

1. Introduction

Human Activity Recognition has seen significant progress in recent years and aims to automate activity recognition from sensory inputs such as cameras or sensors [1]. Although it may seem like a trivial or a relatively simple task to capture on video and detect basic atomic activities performed by humans, it is a very active topic of research within the field of computer vision. This is motivated by the HAR field seeing a useful potential for a great variety

of technology dependent daily life systems. Examples of HAR's potential in the real-world include security surveillance, healthcare surveillance, and even safety monitoring on construction sites [1].

Different approaches to HAR have been tried and tested and today the main suggestion is deep learning models. Combining Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN) models, using 3D CNNs, sequential pose estimation using graph network models such as MediaPipe and openPose, and more recently vision transformers.

HAR continues to be a challenging endeavor not only because of the inherent variety in human motion, but also occlusions, different camera angles and background noise whilst the most relevant spatio-temporal features must be extracted from the data efficiently. This leads to a significant trade-off, common in the computer vision field, between computational efficiency and recognition accuracy which must be leveraged and still remains an obstacle, specially with the adoption of deep learning models that require a vast amount of training data and computation resources. [2]

The suitability of different model architectures for real-time activity recognition and anomaly detection under low-latency constraints remains unclear, making it difficult to determine which approaches are most appropriate for practical deployment in continuous video settings. This is critical, as real-time processing substantially increases both the technical challenge and practical applicability of such systems. The problem is highly relevant due to the widespread availability of surveillance cameras across many domains.

While the ambitious overall goal of the project was originally to compare models based on the task of anomaly detection and measure their inference time for real-time adoption suitability, the scope of this project was simplified to performing action classification, while evaluating performance in terms of both accuracy and computational efficiency. The choices of models and dataset are motivated by both previously tried and tested methods in the current literature as well as a consideration of minimal inference time which is why efficient models have been prioritized and a dataset containing simple atomic activities have been chosen.

The primary target of the project is to classify atomic human activities (i.e., walking, standing, sitting, running) from RGB video inputs using computer vision deep learning architectures with a specific focus on real-time feasibility.

RQ1: Which deep learning architecture, 3DCNN, RNN/LSTM, or Video Vision Transformer, achieves the highest accuracy on atomic activity classification from RGB video?

RQ2: How do these architectures compare in terms of computational efficiency and inference time under real-time constraints?

RQ3: What is the trade-off between recognition accuracy and computational cost across the evaluated models?

The following sections describe the related works, followed by a description of this project's methodology, experiments and results. Finally, a discussion of results is contextualized with the aforementioned research and a mention of limitations is included.

2. Related Work

Due to the success of 2D CNNs in achieving state of the art results for image classification by extracting feature representations and performing a good spatial analysis on static images, these architectures were then transferred and implemented in motion data such as videos for action recognition. But videos presented a different challenge, since they incorporate a third dimension represented by the temporal patterns of the motion.

A few different methods, that tackle in distinct ways the temporal challenge in video data, have emerged throughout the years. They can be divided into the following broader four categories, in order of increased complexity: *two-stream CNNs*, *CNNs combined with RNNs*, *3D CNNs*, and *Transformer based models* [2].

- The two-stream CNNs, first propounded by Simonyan and Zisserman [3], have two networks running in parallel where one is responsible for recognizing the spatial features from the single frames whilst the other captures the temporal features across frames followed by late fusion of these independent networks.
- CNNs combined with RNNs, advanced by Donahue et al. [4], integrate the spatial feature extraction capacities of CNNs with the temporal modeling of RNNs, allowing the CNN layers to extract the relevant spatial features while the RNN learns the temporal ones.
- 3D CNNs, proposed by Ji et al. [5], extend the classical 2D convolution kernel with a third, temporal dimension, allowing it to simultaneously slide through space and time within a stack of consecutive frames, jointly capturing the spatio-

temporal features directly from the video data.

- Transformer-based models dynamically adjust the importance assigned to each part of the input through their self-attention mechanism, allowing them to capture global context and model long-range dependencies, unlike CNNs, which are restricted by local receptive fields. In 2020, Dosovitskiy et al. [6] proposed the Vision Transformer (ViT), which applies this mechanism to images by splitting them into fixed-size patches and treating each patch as a token. Subsequent work extended the transformer architecture to video by incorporating a temporal dimension, enabling the modeling of long-range dependencies across both space and time.

As possibly understood, the field of action recognition is quite diverse, with a broad variety of both preprocessing techniques and different architectures. Advancements continue to be made to this day, especially on which method can best both accurately recognize actions and remain as computationally efficient as possible.

In order to leverage these two important criteria this project focused on three different types of architectures that respectively belong to the three latter categories mentioned. The subsequent sections will describe these in further detail.

2.1. 2D CNN + LSTM

Since the CNN model architecture became the state-of-the-art approach for image classification, it has become a common technique to pair a CNN with a recurrent neural network (RNN). This is because RNN architectures are designed to model sequential input with temporal dependencies, and a video is simply a sequence of images. In this particular study the Long Short-Term Memory (LSTM) network was selected as a type of RNN, motivated by strong classification accuracy of over 94% achieved on the HMDB51 dataset by [7].

A popular approach for both image and video classification tasks, is the use of pretrained models and to opt for a transfer learning techniques. This is especially useful when expressive, large enough and publicly available datasets are hard to come by or computational resources are limited. However, pretrained CNN models can be very deep and large in parameter count, and often very computationally costly. In a 2023 study by [7], they compared several commonly used mobile-oriented pretrained CNN model (ShuffleNet-v2, EfficientNet-b0, MobileNet-v3, and GhostNet) to find which is best at spacial analysis and the most appropriate feature extractor. The results reveal that EfficientNet-b0 consistently outperforms other lightweight CNN's, even achieving comparable or superior performance to ResNet-50, a deeper and more computationally expensive residual network commonly used as a baseline in image classification tasks [8].

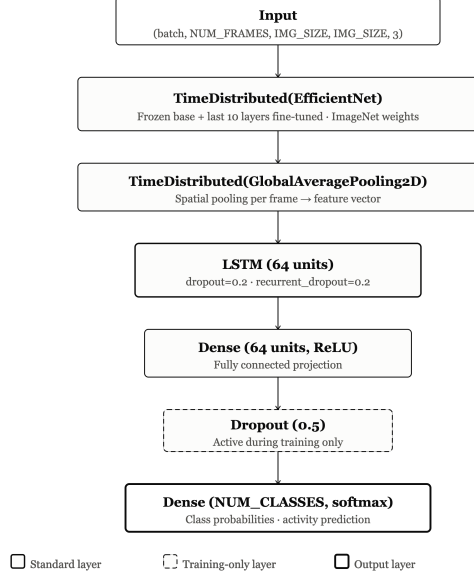


Figure 1: Flowchart of a combination of EfficientNet [9] and LSTM architecture.

2.2. 3D CNN

Building on the general concept of 3D CNNs introduced by Ji et al. [5], Carreira and Zisserman [10] proposed the Inflated 3D ConvNet (I3D) architecture. Rather than building a 3D CNN from scratch, their approach inflated the 2D convolutional and pooling filters of an existing 2D architecture into 3D, allowing the network to be initialized with weights pretrained on large-scale image datasets, before being fine-tuned on video data for action recognition. Which is a major advantage from earlier 3D CNNs models that had to be trained from scratch.

2.2.1. Separable 3D CNN

Unfortunately, 3D CNNs remain computationally expensive so endeavors have been made to develop more efficient models, which were achieved by Xie et al. [11], where the 3D convolutions were replaced with separable convolutions. These separable convolutions factorize the 3D convolution kernel into two kernels that respectively learn the spatial part followed by a separate temporal part.

$$K_{t \times d \times d} \approx K_{1 \times d \times d} * K_{t \times 1 \times 1}$$

Where t is the temporal kernel size and $d \times d$ is the spatial kernel size, the original 3D kernel is approximated by a spatial 2D convolution ($1 \times d \times d$) applied first, followed by a temporal 1D convolution ($t \times 1 \times 1$).

Xie et al. [11] applied this factorization throughout the I3D architecture [10], replacing both the standard convolutional layers and the inception blocks with their separable convolutions, naming the resulting model as S3D. The result was fewer parameters, faster inference and improved accuracy.

Similar works have been developed and their respective authors called their own architectures by different names: “R(2+1)D”, “Pseudo3D network” or “factorized spatio-temporal convolutional networks” but they all implemented this same separable convolution concept [11].

This S3D networks differ from two-stream CNNs since fusion occurs repeatedly after each block throughout the network, rather than only once at the end, as in the original two-stream architecture, where the two separate pathways remain independent until their final outputs are combined.

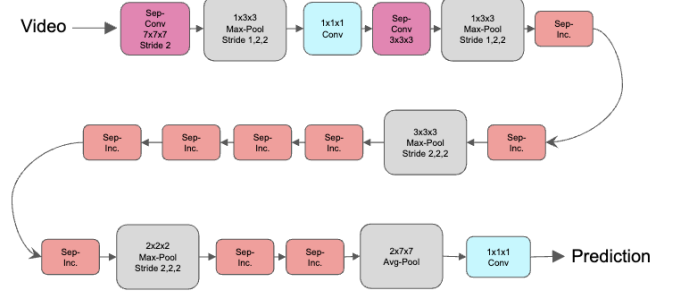


Figure 2: Flowchart of S3D [11].

2.3. Vision Transformer

The original Vision Transformer (ViT) adapts the architecture of the original Transformers [12] as far as possible [6]. Fixed-sized image patches are extracted from the images and linearly embedded before they are handed to the model together with positional embeddings.

2.3.1. Multiscale Vision Transformers

The MViT consists of stages of ViT where, between each stage, the number of channels slowly increases while the spatial resolution decreases [13]. This enables the first few layers to capture bigger spatial features of the image frames while the last layers adapts to the more narrow spatial details. The resolution is reduced through *Multi-Head Pooling Attention* (MHPA) which is a self-attention operator that performs pooling on the latent tensors (the Query, Key and Value’s) spatio-temporal resolution. Similar to pooling in CNNs, the pooling is determined by a kernel size, a stride and a padding and is the cornerstone of the MViT architecture [13]. An overlapping kernel and a size preserving padding are typically applied. Below is the calculation for MHPA where $\mathcal{P}$ represents the pooling operator.

$$\text{PA}(\cdot) = \text{Softmax} \left(\mathcal{P}(Q; \Theta_Q) \mathcal{P}(K; \Theta_K)^T / \sqrt{d} \right) \mathcal{P}(V; \Theta_V)$$

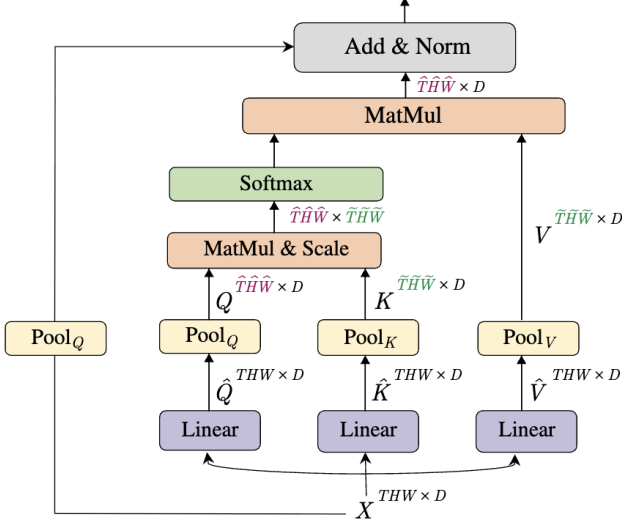


Figure 3: Flowchart of MViT (v1) [13].

Less than a year after the publication of MViT, an updated model, MViT V2, was released [14]. It was set out to address the previous model’s inability to properly handle shift-invariance (the interaction between two patches changed depending on their absolute position even if their relative position did not). *Decomposed Relative Position Embedding* was a way of addressing the issue.

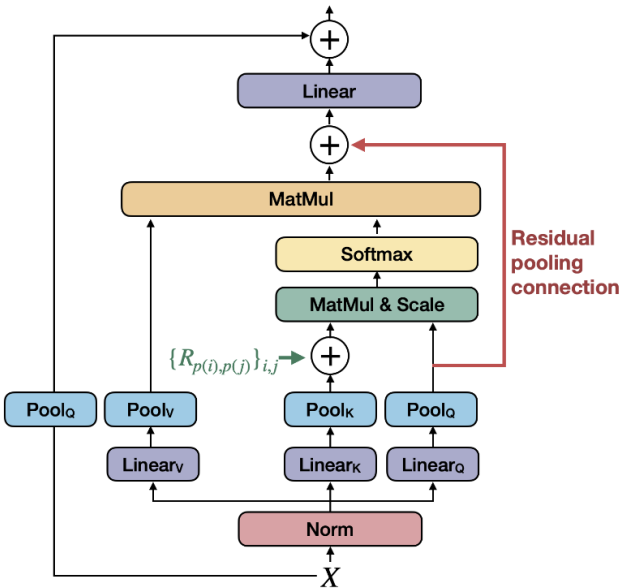


Figure 4: Flowchart of MViT v2 [14].

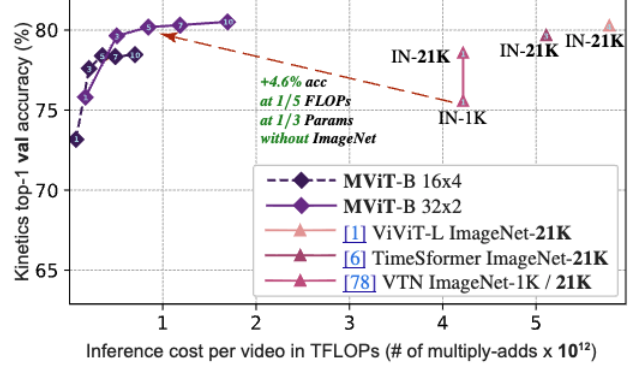


Figure 5: MViT (version 1) accuracy/complexity trade-off on Kinetics-400 for varying number of extracted frames per video compared with previous Video Vision Transformers [13].

MViT had fewer parameters, 26.1 million parameters in the small version and 36.5 million parameters in the base version [13], compared to the base version of the original ViT, which has 87.2 million parameters and gives no opportunity to hand a high resolution frame at the initial stage to the model. The MViT was also trained on the relatively small dataset Kinetics-400 (containing about 240k training videos in 400 classes) and did not need to be pre-trained on a larger dataset (such as ImageNet-21K) compared to the Video Vision Transformer (ViViT) (referring to large version) or the TimeSTransformer base version [13] (see figure 6).

model	pre-train	top-1	top-5	FLOPs × views	Param
Two-Stream I3D [14]	-	71.6	90.0	216 × NA	25.0
ip-CSN-152 [102]	-	77.8	92.8	109 × 3 × 10	32.8
SlowFast 8 × 8 + NL [34]	-	78.7	93.5	116 × 3 × 10	59.9
SlowFast 16 × 8 + NL [34]	-	79.8	93.9	234 × 3 × 10	59.9
X3D-M [33]	-	76.0	92.3	6.2 × 3 × 10	3.8
X3D-XL [33]	-	79.1	93.9	48.4 × 3 × 10	11.0
ViT-B-VTN [84]	ImageNet-1K	75.6	92.4	4218 × 1 × 1	114.0
ViT-B-VTN [84]	ImageNet-21K	78.6	93.7	4218 × 1 × 1	114.0
ViT-B-TimeSformer [8]	ImageNet-21K	80.7	94.7	2380 × 3 × 1	121.4
ViT-L-ViViT [1]	ImageNet-21K	81.3	94.7	3992 × 3 × 4	310.8
ViT-B (our baseline)	ImageNet-21K	79.3	93.9	180 × 1 × 5	87.2
ViT-B (our baseline)	-	68.5	86.9	180 × 1 × 5	87.2
MViT-S	-	76.0	92.1	32.9 × 1 × 5	26.1
MViT-B, 16 × 4	-	78.4	93.5	70.5 × 1 × 5	36.6
MViT-B, 32 × 3	-	80.2	94.4	170 × 1 × 5	36.6
MViT-B, 64 × 3	-	81.2	95.1	455 × 3 × 3	36.6

Figure 6: Given the relative small number of parameters and the lack of pre-training, the MViT (version 1) manage to achieve state of the art performance on the Kinetics-400 [13]. Top-1 means percentage of the test set where the model’s single highest prediction agreed with the label and the Top-5 refers to percentage of predictions where the correct class label was in the top 5 highest predicted classes.

3. Methodology

Considering that the model is intended to be implemented in surveillance scenarios where compute power is scarce and the inference time is important, the model needs to be able to operate on a fairly low number of floating point operations per second (FLOPS). This constraint leaves only a handful of deep learning models left, which are able to still produce high accuracy scores on HAR while having an efficient number of parameters. Video datasets are considered to be one of the more noisy datatypes to handle, given the irrelevant background noise around a subject and the number of frames that might appear (half or completely) empty. The choice of dataset has therefore been based on limiting the background noise while also being a well recognized set in the field of HAR.

3.1. Dataset

While there are multiple datasets that have been produced and tested in the field of HAR, a lot of them consists of videos extracted from Youtube or movies, making the background, angle or lighting often uneasy for the model or multiple people are featured in the clips, which is not preferable. A more noise-free action video dataset with a single person at a time in the frame is harder to find. With these constraints, there were only a few dataset to consider. The KTH dataset, although small, is relatively noise-free, filmed from similar angle over the entire dataset, contains only one subject in the frame at a time and was easily available for download through the official KTH website [15].

The KTH Action Recognition Dataset is from 2004 and contains 6 human activities: walking, running, jogging, boxing, handclapping, and hand waving, and was performed by 25 subjects across 4 scenarios (outdoors, different angles, different clothing, indoors) [15] (see example frames in figure 7). There are a total of 600 videos, each class with 100 videos, recorded at 25 frames per second (fps) with a homogeneous background in black and white. Each video contains four cycles of the action, making it possible to split each video into four clips to create a larger quantity of training examples (2391 clips). The clip length is varying with a mean of 94 and the longest clip containing 350 frames (or 14 seconds).

The models picked are not constructed to handle 350 frames in one given video. Hence, only a portion of the frames from each video was fed to the model (either dense or sparsely extracted depending on model, more on this in the Sampling methods section).



Figure 7: An illustration of 24 example frames of the 6 different classes in the 4 different scenarios from the KTH dataset.

3.2. Pre-processing

Deep learning models need numerical data. Hence, the videos need to be turned into tensors, which is elaborated on in the following section.

3.2.1. General pre-processing

To speed up training time, the frames were extracted from the videos using python's openCV module and permanently stored in directories. A 80%/20% train-test-split with validation set to 20% of the training set was utilized for the experiments.

Each models pre-processing method was imported from Torchvision where, depending on the pre-trained weights of the model and the input size of the model, resizing, cropping, turning to tensors and normalization was applied. The frames were resized (up-scaled) and cropped depending on the expected frame resolution of the model. The normalization used the mean and standard deviation from the dataset that the pre-trained weights had been based on (Kinetics-400 or ImageNet-1K). For example, for EfficientNetB0, the transforms of the model exist in the compose object of the imported model and include resizing each frame to 260 x 260 pixels and then center cropping down to 224 x 224 pixels, which is the expected input size of the model. The frames are also normalized using the mean and standard deviation of the pixel values from the ImageNet-1K dataset that the pre-trained weights was trained on. The model's compose object results in a tensor that perfectly matches the expected input shape.

3.2.2. Sampling methods

When it comes to training the models for HAR on RGB videos, the models expected input tensors imply a sequence of frames of a fixed length (e.g. 16 frames per sample). After a brief exploration of the dataset it is clear that the distribution of frames per samples is far from uniform (see Figure 8).

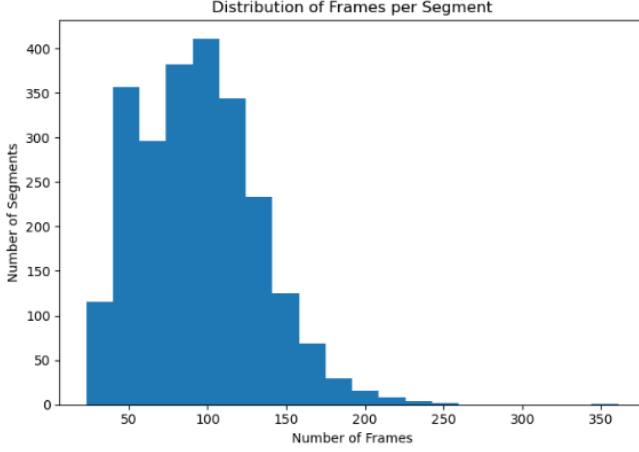


Figure 8: Histogram showing the distribution of frames per video segment

Since the original videos are sampled at 25 fps, this means that each segment is anywhere from a 1 second clip to 14 seconds with the mean being around 4 seconds per action. One important challenge in HAR is the spatial-temporal trade-off, specifically when it comes to sampling. Taking every consecutive frame captures fast motion but it does decrease the temporal receptive field. While sampling sparsely, by taking a fixed number of frames evenly from the entire video duration (e.g. every other frame or every fourth frame), may miss fast motion [16]. Both dense sampling and sparse sampling have been tested and the most consistent and stable training was achieved by utilizing sparse sampling on the hybrid CNN-LSTM model as well as the ViT, while the S3D model saw additional benefits from dense sampling (capturing every consecutive frame within a fixed context window).

3.3. Model Configurations

Bellow follow the configurations used in this study for each of the selected models.

3.3.1. 2D CNN + LSTM

Motivated by the 2023 comparison study of lightweight CNN architectures, the selected model for this approach in this study is the EfficientNetB0 as the backbone feature extractor [7]. EfficientNet is a family of convolutional neural networks designed to achieve high accuracy while keeping computational cost low. Originally developed by Google’s research team, its main idea is compound scaling, where depth (layers), width (channels), and input resolution are scaled together in a balanced way instead of increasing in only one dimension [9]. Architecturally, EfficientNet is built from repeated mobile inverted bottleneck convolution (MBConv) blocks. Each block first expands the number of channels, applies a depthwise convolution, which processes each channel separately rather than mixing them all together, drastically reducing the number of computations compared to a standard convolution, and then compresses the channels

back down. A squeeze-and-excitation step is also applied within each block, which lets the network learn to weight some channels more heavily than others depending on how useful they are for the current input. Together, these design choices let EfficientNet maintain strong accuracy while using far fewer parameters and FLOPs than conventional CNNs [9].

As shown in Figure 9, EfficientNet models (B0–B7) provide a strong accuracy/efficiency trade-off, with EfficientNet-B0 being the smallest and most lightweight baseline in the family.

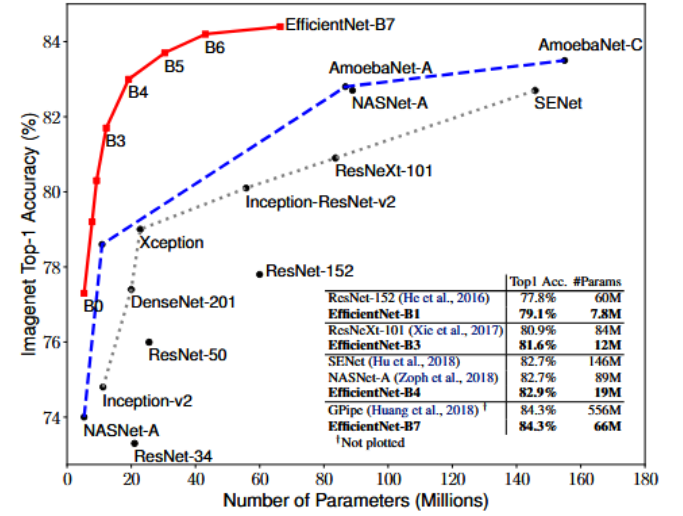


Figure 9: Performance comparison of the EfficientNet family and other common large pretrained CNN models [9].

The EfficientNet-B0 backbone acts as a per-frame feature extractor on the raw video input (tensor of shape [batch, frames, channels, height, width]), producing a sequence of 1280-dimensional feature vectors that are fed into a single-layer LSTM with a hidden size of 256. Extensive experiments were carried out to determine the optimal fine-tuning strategy for the backbone. The results indicated that the most stable training was achieved by unfreezing the last two MBConv blocks (the mobile inverted bottleneck convolution blocks that make up EfficientNet’s architecture) of the EfficientNet model. Loss was computed by the cross entropy formula and optimized using the Adam optimizer, for adaptive learning rates during.

3.3.2. 3D CNN

For the 3D CNN approach, the S3D network described in the earlier section was selected, using the implementation provided by Torchvision with weights pretrained on the Kinetics-400 action recognition dataset. Transfer learning on a large video dataset is important since the S3D has a considerable number of parameters and tends to overfit when trained from scratch on small datasets like the KTH.

Unlike the 2D CNN + LSTM approach, S3D learns the spatio-temporal features through its separable convolutions,

which operate directly on video clips rather than on individual frames. The network receives input tensors of shape [batch, channels, frames, height, width], where each clip consists of a number of selected frames, densely sampled, and posteriorly resized to resolution of 224×224 , matching pre-processing expected by the pretrained model. During training, the starting frame of each clip is selected randomly as a form of temporal augmentation, while at evaluation time a deterministic center selection is used. Clips shorter than the selected frame are looped cyclically up to the required length rather than discarded.

To adapt the pretrained network to the six KTH action classes, only the final classification head was replaced. A two-stage fine-tuning strategy was adopted: the convolutional backbone was initially frozen so that only the newly initialized head was trained, allowing it to first stabilize. After a fixed number of epochs, the full backbone was unfrozen and the entire network was fine-tuned end-to-end. Upon unfreezing, the optimizer was reinitialized with a learning rate reduced by a factor of ten, in order to preserve the pretrained temporal features while still allowing them to adapt to the target domain.

The training followed the original S3D paper configurations using stochastic gradient descent with a momentum of 0.9 and weight decay. A staged learning rate decay schedule was applied, reducing the learning rate by a factor of ten at fixed intervals.

3.3.3. Vision Transformer

Vision transformers expect sequential inputs. MViT uses patch embedding to turn patches of the video frames into sequences of tokens. The patch embedding is built into Pytorch’s (Torchvision) official implementation of MViT v2, where kernel size, stride and padding can be given as arguments [17]. Pre-trained weights trained on the Kinetics-400 were imported from Torchvision and given as an argument to the model. 16 frames per video were used during the creation of the pre-trained weights of Kinetics-400 [18], which is why the same number frames was used during the transfer learning to the KTH dataset. AdamW was applied as the optimizer during training in accordance with the original implementation [14], and Cross Entropy Loss was used as the loss function. The head had to be adapted to KTH’s 6 action classes and the second last layer was too unfrozen for better generalization on the new dataset. These layers were trained for 30 epochs with a learning rate of 10^{-3} for the second last layer, and 10^{-4} for the last layer. 7.1M out of 34.2M parameters were trainable during the training iteration (see table 1).

4. Experiments and Evaluation

The following section describes the experimental setup, as well as the training and evaluation approach for this study.

4.1. Experimental setup

In order to carry out the experiments this project relied on the following hardware and environments:

- GPU: Nvidia A100
- Python v3.12.0
- Pytorch v2.11.0
- Torchvision v0.26.0

For additional reliability of the results, the environments were made deterministic by setting global seeds to control the inherent randomness of the different modules. After having set up a deterministic environment, the experiments are set up with some fixed independent variables with the hope of more valid results. This included all model architectures learning on the same context length (number of frames per sample), as well as 30 epochs per model, although with early stopping to prevent overfitting. Table 1 summarizes the experimental setup in terms of each model and its setup.

TABLE 1: Pre-trained weights, # of trained parameters, epochs of training and # of frames used.

Model	Pre-trained	Trained parameters	Epochs	# of frames
EfficientNet-LSTM	ImageNet-1K	2,705,846	30*	16
S3D	Kinetics-400	8,320,048†	30	16
MViT v2 (small)	Kinetics-400	7,104,774	30	16

* = Early stopping was used for the EfficientNet-LSTM model.

† = Backbone frozen for epochs 1–14, full network unfrozen from epoch 15.

4.2. Evaluation metrics

To evaluate the models, several evaluation metrics were used. Accuracy, precision, recall and F1-score was computed on the reserved test set to confirm each models prediction performance and generalization capacity. Floating point operations per second (FLOPS) and inference time was measured to estimate the models latency for real-world application.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{FLOPS} = \frac{\text{Number of Floating Point Operations}}{\text{Time (seconds)}}$$

Inference time was computed using the time module in Python, by synchronizing the GPU clock with the Python time module, using Pytorch’s Cuda module. Then a single dummy sample of batch size 1, is fed to the model in evaluation mode (no gradient computations) and the inference

time is computed as the difference in input time to output time.

5. Results and Discussion

This section presents the quantitative results of the experiments for each model architecture. Followed by a discussion of the obtained results, summarizing the results of the models in terms of both the above mentioned evaluation metrics as well as a comparative analysis and a discussion of the limitations to this study.

5.1. Quantitative results

Table 2 contains the results of our selected models with a 16 frame context length comparison. S3D, pre-trained on Kinetics-400 achieved a 96.5% average accuracy over KTH dataset’s 6 classes, which turned out to be our most accurate model. The most efficient model in terms of GFLOPS and inference time was the combination of EfficientNet base and LSTM with 6.41 GFLOPS and an inference time of 5.72 (on a single Nvidia A100 GPU).

TABLE 2: Accuracy, GFLOPS and latency of compared models with 16 frame context length.

Model	Overall accuracy	Parameters	GFLOPS	Inference time*(ms)
Effic.Net-LSTM	86%	5,584,002	6.41	5.72
S3D	96.5%	8,320,048	18.20	7.29
MViT v2	92.7%	34,537,744	64.46	23.75

* = Inference time of one forward pass of a single sample using a Nvidia A100.

5.1.1. EfficientNet + LSTM

Although certain measures were taken to attempt to minimize overfitting, such as adding dropout layers, early stopping during training and controlling the complexity of the model, figure 10 shows that there is some overfitting towards the last epochs in the training process. It is evident by noticing the training loss decreasing consistently until the last epoch, while the validation loss plateaus and starts to increase suddenly towards the end.

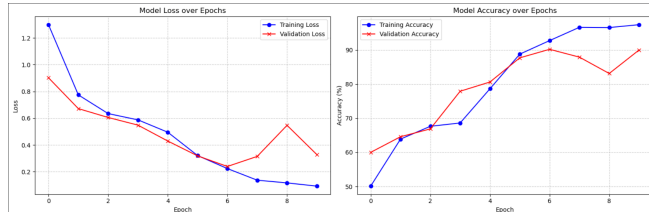


Figure 10: The plot on the right side shows the training loss and validation loss across all epochs. On the left, the plot shows the training and validation accuracy across the epochs.

The model achieved 89% accuracy on the test set with a clip length of 24 frames per input sample and 86% accuracy

on the test set with a clip length of 16 frames (which greatly reduces the GFLOPS of the model due to fewer parameters and smaller input tensors). However, as can be seen in figure 11 there is a clear indication that the model struggles with fine-grain temporal-spatial dependencies. Specifically, it wrongly classifies a good number of jogging samples as running and vice versa.

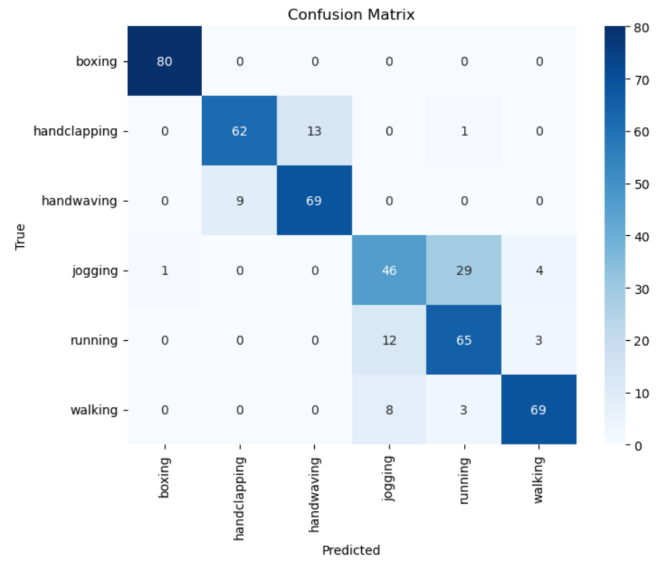


Figure 11: Confusion matrix for the EfficientNetB0 + LSTM model with 16 frame context length

5.1.2. S3D

The S3D was the strongest performing model in this study across every context length tested. As shown in Table 3, accuracy improved consistently as the temporal context was extended, rising from 96.5% at 16 frames to 97.9% at 32 frames and reaching 98.5% at 64 frames, with the F1-score following the same trend (96%, 98% and 99% respectively).

TABLE 3: S3D model context length, GFLOPS, accuracy and F1-score results

Context Length	GFLOPS	Accuracy	F1-Score
16 frames	18	96.5%	96%
32 frames	36	97.9%	98%
64 frames	72	98.5%	99%

This indicates that the model performs better with longer temporal context, as the additional frames provide more motion information for the separable convolutions to learn from. This improvement comes at a computational cost, with GFLOPS increasing linearly from 18 at 16 frames to 72 at 64 frames.

The confusion matrix for the 16-frame configuration, shown in Figure 12, indicates that the main source of error is the confusion between the jogging and running classes, with 5 jogging samples predicted as running and 5 running samples predicted as jogging. This exhibits a similar behavior

observed in the EfficientNet + LSTM and MViT v2 models, and reflects the inherent difficulty of distinguishing these two actions, which differ mainly in motion speed rather than in their spatial appearance. The remaining errors are minor: 6 handwaving samples are misclassified as handclapping and a single walking sample is predicted as boxing, while the handclapping class is classified perfectly.

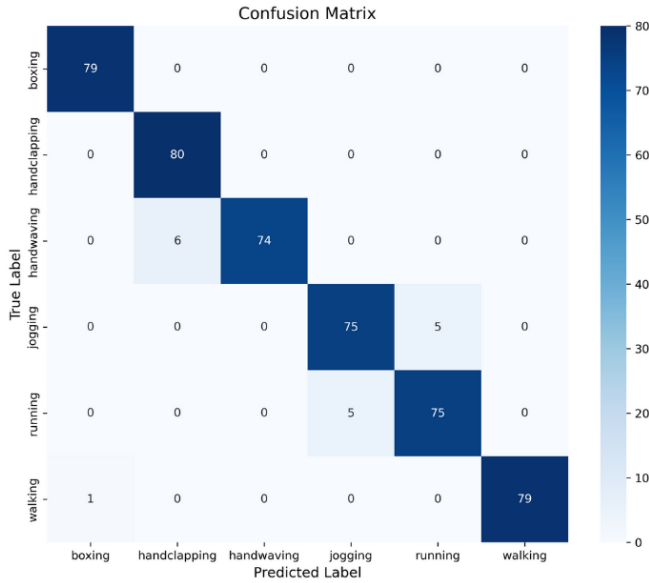


Figure 12: Confusion matrix of the results from the S3D with 16 frame context length

To note that the jogging/running confusion is resolved as the temporal context is increased, which is observable on the accuracy gained at the 32 and 64 frame configurations, longer clips capture the motion patterns that separate the two movements more reliably than a 16 frame context.

The parameter count of the model (8.3 million) combined with its high accuracy indicates that the Kinetics-400 pretraining transferred well to the KTH domain, despite the difference in scale, background and angle variations between the two datasets.

5.1.3. Vision Transformer

The overall accuracy of the MViT v2 (small) model on the 6 classes of the KTH dataset was 92.7%. It sees some difficulty in differentiating 'jogging' and 'running' just as the compared models did (see figure 13).

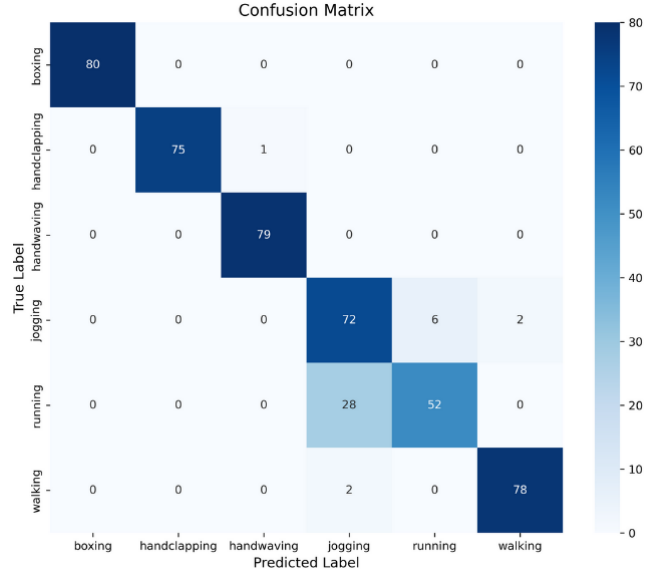


Figure 13: Confusion matrix of the results from the MViT V2 (small) with 16 frame context length.

5.2. Discussion

The goal of this project was to discover the most suitable model architecture and methods for computationally efficient HAR, that would be feasible for real-time inference on edge devices (i.e. raspberry-pi, smartphone, etc.). As seen in Table 2, all 3 models performed reasonably well, still with the hybrid CNN/LSTM approach lacking behind on accuracy. However, it is the lightest model with the fewest parameters and lowest GFLOPS at only 6.41 compared with the 18.2 of the S3D and a massive 64.46 of the MViT. The hybrid CNN and LSTM architecture performed overall similarly to what has been achieved in previous literature, although lacking behind on the performance when using smaller context lengths, with [7] reaching above 94% accuracy on all RNN variants in combination with the EfficientNet using only 15 frames per sample. In comparison, this study was able to achieve 89% test accuracy with 24 frames and 86% with 16 frames, suggesting that with the KTH dataset, this hybrid model benefits from more context.

In contrast to the CNN-LSTM approach, the S3D model emerged as the most robust architecture for this specific task, achieving a superior overall accuracy of 96.5%. This performance directly validates the findings of [11], who originally proposed replacing standard 3D convolutions with separable convolutions to factorize the spatial and temporal kernels. As theorized in the literature, this factorization yields fewer parameters and faster inference while improving overall accuracy. Operating at 18.20 GFLOPS with a 7.29 ms inference time on the A100 GPU, these results confirm that S3D effectively captures fine-grained spatio-temporal features within a limited 16-frame context window far better than 2D CNN-RNN approaches.

While the Multiscale Vision Transformer (MViT v2) achieved a highly respectable 92.7% accuracy, its perfor-

mance must be evaluated against the project’s primary objective, which was real-time feasibility. In this context, the model’s massive computational cost emerges as its critical flaw. Demanding 64.46 GFLOPS, more than triple that of the S3D model and ten times that of the EfficientNet-LSTM, the MViT v2 appears fundamentally incompatible with low latency edge deployment. This high FLOP count is an inherent byproduct of its architecture, as noted in the literature [2], the self-attention mechanisms required to dynamically capture global context across video patches are incredibly computationally expensive. Furthermore, because Transformers lack the restricted local receptive fields [2], they naturally require vast amounts of data to learn these representations effectively [6]. Therefore, while the MViT v2 is undeniably powerful, its heavy architectural footprint proves that high accuracy does not equate to practical utility in compute constrained environments.

Ultimately, these results shine light on the critical trade-off between model accuracy and practical real-time feasibility. While all three models demonstrated excellent sub-30 ms inference times on an Nvidia A100 GPU, these raw latency numbers do not perfectly translate to edge deployment. Processing frames at 64.46 GFLOPS (MViT v2) on a low-power edge device would likely result in immediate thermal throttling. Therefore, the choice of architecture becomes situational, for deployments with extreme power constraints, the EfficientNet-LSTM’s 6.41 GFLOPS makes it an acceptable compromise. However, for most standard real-time HAR applications, the S3D architecture hits the definitive sweet spot, and is the most promising approach for real-time HAR goals.

5.3. Limitations

The biggest and most obvious limitation of this study, is the dataset. Although the dataset was deemed as most suitable for these experiments it does not come without its shortcomings. When sampling frames from the dataset of such various length video segments, it leads to a loss of temporal context in some of the samples. Specifically, when some videos contain well over 200 frames but the models are only exposed to 16 of those frames. The original videos are also quite old and not of good quality for today’s standards, being black and white, with a resolution of only 120x160 pixels. However, perhaps the most important limitation is the similarity between the jogging and running classes. Not only did the models most commonly struggle with detecting the specific differences in those two actions, but upon further inspection, they were hard to distinguish with the human eye as well.

Lastly, the study did not extend its methods to model tuning, including hyperparameter optimization and cross validation, and also due to time constraints the experiments were not repeated.

6. Conclusion and Future Work

Computer vision and HAR are fields in full movement, offering immense application opportunities but continuing to face obstacles such as the scarcity of labeled, non-noisy video datasets. This data limitation was particularly evident in our findings regarding newer architectures, while the MViT v2 is powerful, its massive data requirements and high computational cost make it unsuitable for compute constrained environments. Ultimately, evaluating these models for real-time edge deployment requires balancing inference speed with reliable performance. While the EfficientNet-B0 LSTM hybrid achieved the lowest computational footprint (6.41 GFLOPS and 5.72 ms inference time), the S3D architecture emerged as the definitive recommendation of this study. Achieving the highest accuracy of 96.5% across the KTH dataset while maintaining a highly manageable FLOP count, S3D provides the optimal speed-accuracy trade-off required for practical, real-time video surveillance.

6.1. Future work

Once time gives the opportunity, there is a lot of future work that can improve our efforts. The main future focus is to turn the task of the models into anomaly detection. Labeling the KTH dataset action classes as normal behavior and outliers as anomalies, could be a potential solution. Measuring the models prediction certainty would be another way.

The scarcity of datasets leaves a natural space for custom data collection. This way, it would also be possible to acquire full control of the attributes of the dataset such as using a pale background and one common camera angle.

Shuffling the frames and comparing the accuracy with the original models would reveal if the models are utilizing the order of the frames, which was something that [13] applied to the original ViT [6] and resulted in no performance decay, indicating that the model was not leveraging the order of the frames. Applying these approaches to the models, especially the MViT, could give an indication of an improvement potential. The opportunity for higher measured accuracy could also potentially be obtained if more extensive hyper-parameter tuning were applied.

References

- [1] F. Naik, H. Rahman, M. Shaheen, A. Algarni, N. A. Almujaal, and A. Jalal, “A review of video-based human activity recognition: theory, methods and applications,” *Multimedia Tools and Applications*, July 2024.
- [2] K. Alomar, H. I. Aysel, and X. Cai, “Cnns, rnns and transformers in human action recognition: a survey and a hybrid model,” *Artificial Intelligence Review*, vol. 58, no. 12, pp. 1–44, 2025.
- [3] K. Simonyan and A. Zisserman, “Two-stream convolutional networks for action recognition in videos,” *Advances in neural information processing systems*, vol. 27, 2014.
- [4] J. Donahue, L. Anne Hendricks, S. Guadarrama, M. Rohrbach, S. Venugopalan, K. Saenko, and T. Darrell, “Long-term recurrent convolutional networks for visual recognition and description,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 2625–2634.

- [5] S. Ji, W. Xu, M. Yang, and K. Yu, "3d convolutional neural networks for human action recognition," *IEEE transactions on pattern analysis and machine intelligence*, vol. 35, no. 1, pp. 221–231, 2012.
- [6] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," *arXiv preprint arXiv:2010.11929*, 2020.
- [7] S. N. Moutsis, K. A. Tsintotas, I. Kansizoglou, and A. Gasteratos, "Evaluating the performance of mobile-convolutional neural networks for spatial and temporal human action recognition analysis," Dec 2023, doi: <https://doi.org/10.3390/robotics12060167>.
- [8] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," Dec 2015, <https://arxiv.org/abs/1512.03385>.
- [9] M. Tan and Q. V. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," Sep 2020, <https://arxiv.org/pdf/1905.11946>.
- [10] J. Carreira and A. Zisserman, "Quo vadis, action recognition? a new model and the kinetics dataset," in *proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 6299–6308.
- [11] S. Xie, C. Sun, J. Huang, Z. Tu, and K. Murphy, "Rethinking spatiotemporal feature learning: Speed-accuracy trade-offs in video classification," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 305–321.
- [12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [13] H. Fan, B. Xiong, K. Mangalam, Y. Li, Z. Yan, J. Malik, and C. Feichtenhofer, "Multiscale vision transformers," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 6824–6835.
- [14] Y. Li, C.-Y. Wu, H. Fan, K. Mangalam, B. Xiong, J. Malik, and C. Feichtenhofer, "Mvitv2: Improved multiscale vision transformers for classification and detection," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 4804–4814.
- [15] C. Schudt, I. Laptev, and B. Caputo, "Recognizing human actions: a local svm approach," in *Proceedings of the 17th International Conference on Pattern Recognition, 2004. ICPR 2004.*, vol. 3. IEEE, 2004, pp. 32–36.
- [16] L. Wang, Y. Xiong, Z. Wang, Y. Qiao, D. Lin, X. Tang, and L. V. Gool, "Temporal segment networks for action recognition in videos," Nov. 2019.
- [17] "Torchvision 0.26 documentation." [Online]. Available: <https://docs.pytorch.org/vision/0.26/modules/torchvision/models/video/mvit.html>
- [18] "Torchvision 0.26 documentation." [Online]. Available: <https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.video.mvitv2s.htmltorchvision.models.video.mvitv2s>